

We Can Describe Any Finite Behavior Using an FSM

What We Know So Far

For a synchronous sequential system,

Current State + Input $\longrightarrow$ Next State + Output

Using this idea, we can design machines for many different behaviors:

Sequence
Detector

Traffic
Controller

Vending
Machine

Serial
Adder

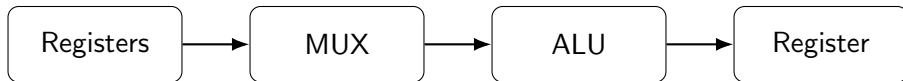
Next Question

If every problem requires a different behavior, do we need to design a different FSM and circuit for every problem?

Can We Use the Same Circuit for Different Problems?

Yes!

The same hardware components can be reused for different computations.



For example, the same ALU can perform $A + B$, $A - B$, $A \wedge B$, $A \vee B$ by changing its **control inputs**.

So we do not need a completely new circuit

We can reuse the same datapath and select different operations, sources, and destinations.

But...

Who changes these control signals, and in what sequence, for each new problem?

What Changes When the Problem Changes?

Same hardware, different problems



Problem 1

$$C = A + B$$

ADD → Store

Problem 2

$$C = (A + B) - D$$

ADD → Store → SUB → Store

The hardware is reusable, but the **control sequence must change**.

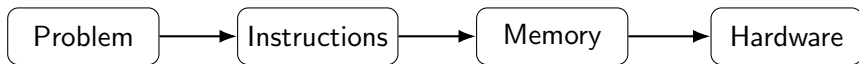
Can we change this sequence without changing the circuit?

How Can We Avoid Changing the Circuit Manually?

Instead of redesigning the control circuit for every problem,

Key Idea

Represent the required operations as **instructions**.



ADD	STORE	SUB	STORE
-----	-------	-----	-------

Same hardware + different instructions $\Rightarrow$ different computation

Instructions Are Stored in Memory

The sequence of operations can itself be stored in **memory**.

ADD R1, R2, R3	Instruction 1
STORE R1, ...	Instruction 2
SUB R4, R1, R5	Instruction 3
STORE R4, ...	Instruction 4

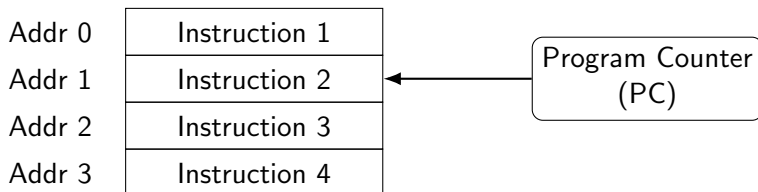
Stored-Program Idea

Instead of building the sequence into the circuit,
store the sequence as instructions in memory.

But now the hardware must know:
Which instruction should be executed next?

How Does the Hardware Know Which Instruction Comes Next?

Each instruction is stored at some memory location.



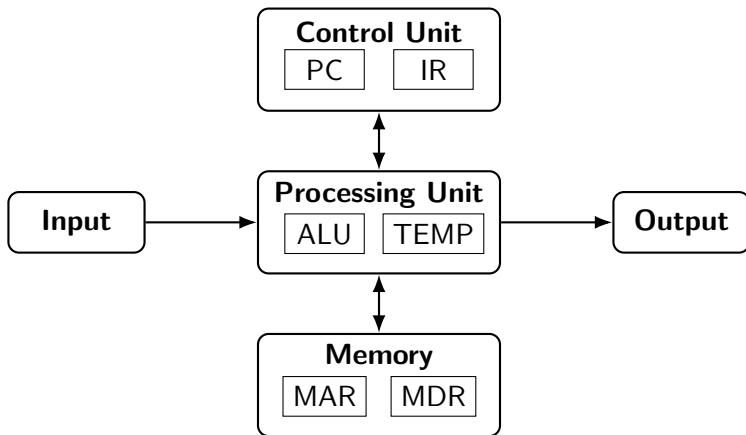
Program Counter

The **PC** stores the address of the next instruction to be executed.

Normally, $PC \leftarrow PC + 1$ (LC-3)
after an instruction is fetched.

So instructions can be executed sequentially from memory.

The Von Neumann Model

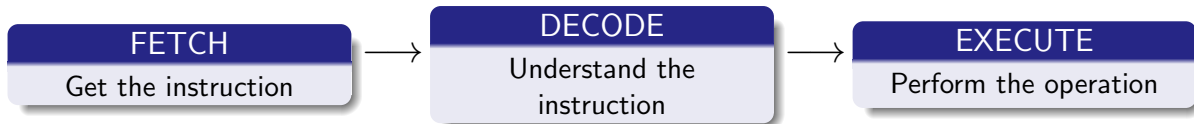


Stored-Program Concept

Instructions and data are both stored in memory.

How Does the Machine Execute a Program?

Instructions are stored in memory, and the PC identifies the next instruction to be processed.



Instruction Processing

The processor repeatedly performs

FETCH → DECODE → EXECUTE

What exactly happens inside these steps?

A Closer Look at the Instruction Cycle

The execution of an instruction can be viewed as a sequence of phases.

1. FETCH

Fetch the instruction

2. DECODE

Decode the instruction

3. EVALUATE ADDRESS

Determine the memory address

4. FETCH OPERANDS

Obtain the source operands

5. EXECUTE

Perform the operation

6. STORE RESULT

Write the result

Important

Not every instruction requires all six phases.

What Is an Instruction?

A program is a sequence of **instructions** stored in memory.

An instruction tells the processor **what operation to perform**.

ADD R1, R2, R3

$R1 \leftarrow R2 + R3$

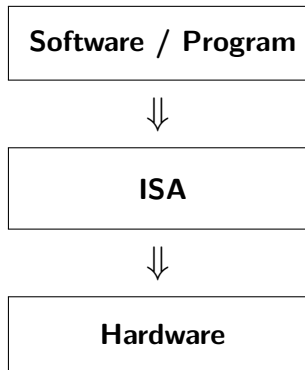
But we need some rules:

Which operations are allowed? How are they represented?

Instruction Set Architecture (ISA)

Instruction Set Architecture (ISA)

ISA is the interface between software and hardware.



ISA specifies:

- Instructions
- Programmer-visible registers
- Instruction formats
- Addressing modes
- Data types

Software sees the ISA, not the internal hardware implementation.

ISA vs. Microarchitecture

Two different questions about a computer:

ISA

WHAT does the machine provide?

- Instructions
- Registers
- Addressing modes
- Instruction formats

Microarchitecture

HOW is the ISA implemented?

- Datapath
- ALU
- Registers
- Control FSM

One ISA can have different microarchitectures.

From Program to Hardware

Program / Software



**Instruction Set Architecture
(ISA)**



Microarchitecture



Logic: Gates, Registers, FSMs



Devices: Transistors

ISA

What the programmer sees

Microarchitecture

How the ISA is implemented

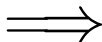
LC-3: Our Example

To understand how an ISA is actually implemented, we will use a simple computer:

LC-3

LC-3 ISA

Instructions
Registers
Instruction formats
Addressing modes

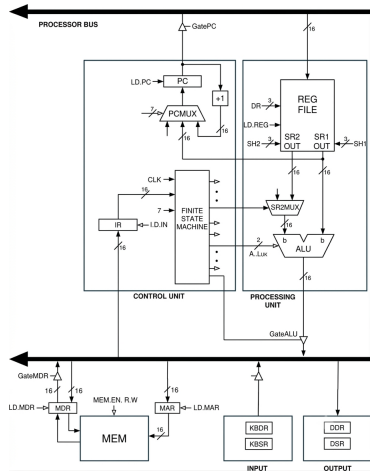


LC-3 Microarchitecture

Datapath
ALU
Registers
Control FSM

How does the hardware execute an LC-3 instruction?

LC-3 Microarchitecture: Datapath + Control

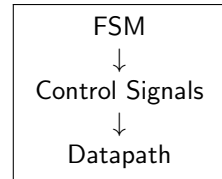


Datapath

PC, IR, Registers, ALU,
MAR, MDR, Memory

Control

A finite state machine generates the
required control signals.

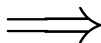


The Control Unit Is an FSM

The LC-3 control unit can be described as a **Finite State Machine**.

Inputs to the FSM

Instruction bits (IR)
Condition information
Memory status



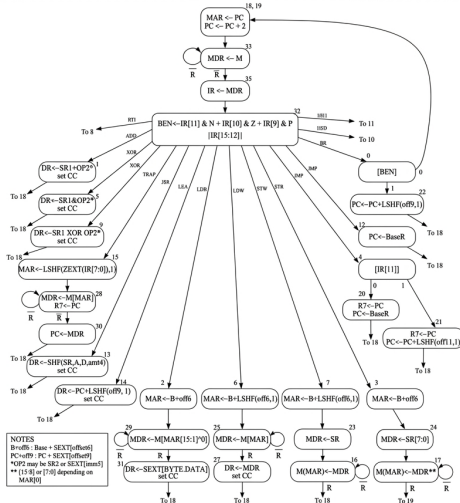
Outputs from the FSM

Load signals
MUX select signals
Gate / Memory signals

Each state specifies the **control signals** required for that step of instruction processing.

Fetch → Decode → Instruction-specific states

LC-3 Control State Diagram



Start here

Fetch the instruction



Decode the instruction



Follow the path for the
decoded instruction

Examples

ADD, LDR, STR, BR

Then return to FETCH

LC-3: Fetching an Instruction

Before executing any instruction, it must first be **fetch**ed from memory.

Step 1

$$MAR \leftarrow PC$$

Step 2

$$MDR \leftarrow M[MAR]$$

Step 3

$$IR \leftarrow MDR$$

$$PC \leftarrow PC + 2 \text{ (LC-3b)}$$

Read the instruction from
memory.

Load the instruction into the
IR.

Send the instruction address to
memory.

The instruction is now inside the processor and is ready to be **decoded**.

LC-3: Decoding the Instruction

After FETCH, the instruction is available in the **Instruction Register (IR)**.

IR[15:12]	IR[11:0]
Opcode	Remaining instruction bits

The **opcode** tells the control unit which instruction has been fetched.

ADD LDR STR BR ...

Opcode → Control FSM → Instruction-specific path

Example: The ADD Instruction

ISA View

ADD R1, R2, R3

Meaning:

$$R1 \leftarrow R2 + R3$$

Instruction Format

15 : 12	11 : 9	8 : 6	2 : 0
Opcode	<i>DR</i>	<i>SR1</i>	<i>SR2</i>

(Immediate mode fields omitted for simplicity.)

Executing the ADD Instruction

ADD R1, R2, R3

$$R1 \leftarrow R2 + R3$$



Read source registers



Perform addition



Write destination register

No data-memory access is required.

What If the Operand Is in Memory?

For ADD: operands are in registers

$R2, R3 \longrightarrow \boxed{\text{ALU}}$

But an operand may be in memory.

$\boxed{\text{Memory}} \longrightarrow \boxed{\text{Register}}$

Which memory location should be accessed?

Example: `LDR R1, R2, #5` $R1 \leftarrow M[R2 + 5]$

Addressing Modes

How does an instruction specify where an operand is located?

This is determined by its **addressing mode**.

Addressing Mode	Operand / Address Comes From
Immediate	Instruction itself
Register	Register
PC-Relative	PC + offset
Indirect	Memory through a pointer
Base + Offset	Base register + offset

LC-3 supports all five of these addressing modes.

1. Immediate Addressing

The operand is contained in the instruction.

Example:

ADD R1, R2, #5

Meaning:

$$R1 \leftarrow R2 + 5$$



No memory access is required to obtain the operand.

2. Register Addressing

The operand is stored in a register.

Example:

ADD R1, R2, R3

Meaning:

$$R1 \leftarrow R2 + R3$$



The instruction specifies the register containing the operand.

3. PC-Relative Addressing

The memory address is calculated relative to the PC.

Example:

LD R1, LABEL

Effective Address:

$$EA = PC + \text{SEXT}(PCoffset9)$$

Then:

$$R1 \leftarrow M[EA]$$

PC-Relative Addressing: Example

x3000: LD R1, VALUE

x3001: ...

x3005: VALUE: .FILL #25

After fetch:

$$PC = \text{x3001}$$

$$PCoffset9 = \text{x3005} - \text{x3001} = 4$$

$$EA = PC + PCoffset9 = \text{x3001} + 4 = \text{x3005}$$

$$R1 \leftarrow M[\text{x3005}] = 25$$

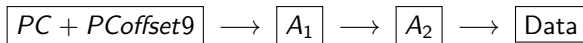
4. Indirect Addressing

Example: LDI R1, LABEL

$$A_1 = PC + \text{SEXT}(PCoffset9)$$

$$A_2 = M[A_1]$$

$$R1 = M[A_2]$$



**The first memory access gives an address,
the second memory access gives the data.**

Indirect Addressing: Example

x3000: LDI R1, PTR

x3005: PTR: .FILL x4000

x4000: .FILL #25

After fetch: $PC = x3001$

$$PC_{offset9} = x3005 - x3001 = 4$$

$$A_1 = x3001 + 4 = x3005$$

$$A_2 = M[x3005] = x4000$$

$$R1 = M[x4000] = 25$$

5. Base + Offset Addressing

Example: LDR R1, R2, #5

$$EA = R2 + \text{SEXT}(\text{offset6})$$

Suppose:

$$R2 = \text{x4000} \quad \text{and} \quad \text{offset6} = 5$$

$$EA = \text{x4000} + 5 = \text{x4005}$$

$$\text{If } M[\text{x4005}] = 25,$$

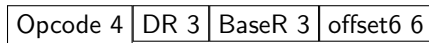
$$R1 \leftarrow M[\text{x4005}] = 25$$

Why Do We Need an Offset?

LC-3 instruction size = **16 bits**

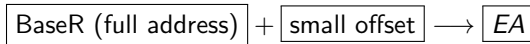
A memory address is also **16 bits**.

For LDR:



There is no space for a complete 16-bit address.

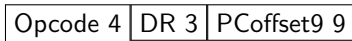
Instead:



$$EA = BaseR + SEXT(offset6)$$

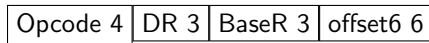
Why Different Offset Sizes?

PC-relative: LD



$$EA = PC + \text{SEXT}(PCOffset9)$$

Base + Offset: LDR



$$EA = BaseR + \text{SEXT}(offset6)$$

More fields in the instruction $\Rightarrow$ fewer bits available for the offset
--

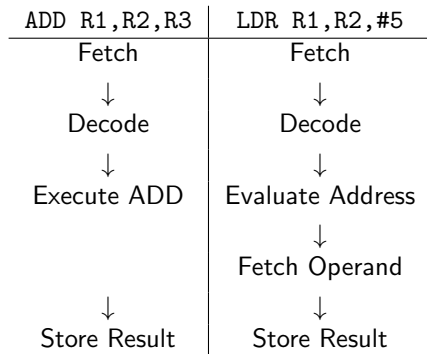
LDR Through the Instruction Cycle

LDR R1, R2, #5

Phase	Operation
Fetch	Instruction $\rightarrow$ IR
Decode	Identify LDR
Evaluate Address	$EA = R2 + \text{SEXT}(5)$
Fetch Operand	$M[EA] \rightarrow MDR$
Store Result	$MDR \rightarrow R1$



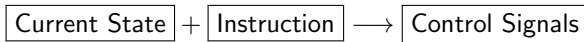
Different Instructions, Different Paths



Opcode determines the path taken by the Control FSM

Who Generates the Control Signals?

The **Control Unit** is an FSM.



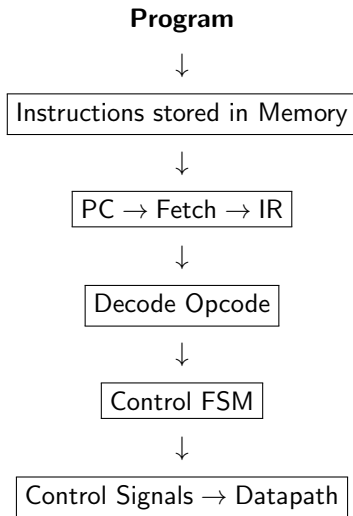
For example:

LDR

Fetch → Decode → Address → Memory Read → Register Write

Each state activates the required datapath signals.

Putting Everything Together



ISA = What

Microarchitecture = How

From One Instruction to the Next

A program is a sequence of instructions.



After execution,

PC points to the next instruction

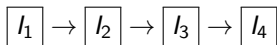
Normally: next sequential instruction

Control-flow instruction: PC is changed

BR, JMP, JSR, ...

Control-Flow Instructions

Normally, instructions execute sequentially:



But programs need:

Decisions Loops Function Calls

These require changing the PC:

$PC \leftarrow \text{new instruction address}$

LC-3 examples: BR, JMP, JSR

Conditional Branch: BR

BRp LOOP

The processor checks the condition codes:

<i>N</i>	<i>Z</i>	<i>P</i>
----------	----------	----------

If the specified condition is true:

$$PC \leftarrow PC + \text{SEXT}(PCoffset9)$$

If the branch is not taken:

PC already points to the next sequential instruction
--

Thus, BR selects which instruction is fetched next.

Example: How a Loop Works

LOOP: ADD R1,R1,#-1

BRp LOOP

If the result is positive:

$$P = 1 \Rightarrow PC \leftarrow \text{LOOP}$$

The processor fetches the ADD again.

$$\text{ADD} \rightarrow \text{BRp} \rightarrow \text{ADD} \rightarrow \dots$$

When $P = 0$, execution continues to the next instruction.

How Does BR Know the Result?

LC-3 maintains three condition codes:

<i>N</i>	<i>Z</i>	<i>P</i>
----------	----------	----------

They describe the result of the most recent instruction that **updates the condition codes**:

N : Negative *Z* : Zero *P* : Positive

For example:

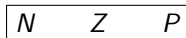
ADD R1,R1,#-1

updates the condition codes.

Then BR_p LOOP checks: $P = 1$?

Condition Codes Are Processor State

The processor must remember the result category:



When an instruction **updates the condition codes**:

$\text{Result} < 0 \Rightarrow N = 1$

$\text{Result} = 0 \Rightarrow Z = 1$

$\text{Result} > 0 \Rightarrow P = 1$

Exactly one of N, Z, P is set.

Instruction Result $\rightarrow$ Condition Codes $\rightarrow$ BR

What Have We Built?

We started with:

Hardwired FSM + Datapath $\rightarrow$ Fixed Behavior

Now we have:

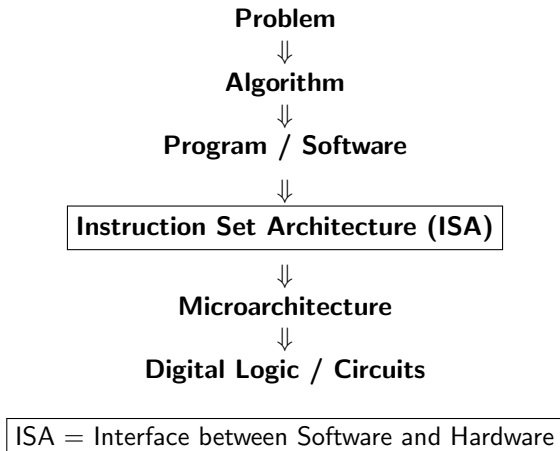
Instructions in Memory



Control FSM $\rightarrow$ Datapath

Different instructions cause different sequences of hardware operations.

Where Does the ISA Fit?



How Complex Should the Hardware Be?

We have defined the **ISA**.

But an ISA can contain:

Simple instructions or **Complex instructions**

For example:

ADD

vs.

Matrix Multiply

Who should perform the complex work?

Software

or

Hardware?

Simple vs. Complex Instructions

Simple Instruction

Performs a small primitive operation

ADD XOR LOAD

Complex Instruction

Performs a larger amount of work

String Copy
Matrix Multiply

Instruction complexity influences where the implementation burden lies
--

Complex vs. Simple Instructions: Trade-off

Complex Instructions

- Smaller code size
- Better memory utilization
- Easier HLL mapping
- More work for implementation

Simple Instructions

- More primitive operations
- More work for compiler
- More optimization freedom
- Simpler ISA semantics

Where should the optimization burden lie?

Semantic Gap: Who Does the Work?

High-Level Language (HLL)

Simple ISA

Large semantic gap

Harder HLL $\rightarrow$ ISA mapping

More burden on Software

Complex ISA

Small semantic gap

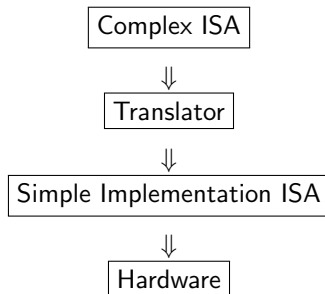
Easier HLL $\rightarrow$ ISA mapping

More burden on Implementation

Semantic gap influences where the design burden lies

Can We Have Both?

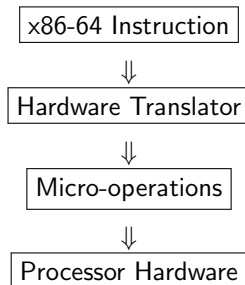
A complex ISA can be translated into simpler internal operations.



Translator can be implemented in **software or hardware**.

Complex ISA, Simple Internal Operations

Example: **x86-64 processors**



**Software sees x86-64,
hardware can execute simpler internal operations.**

From ISA to Microarchitecture Design

The ISA tells us **what** each instruction must do.



To implement the ISA, we must decide:

- What hardware resources are required?
- Which resources can be shared?
- How many operations happen in one clock cycle?
- How are the operations controlled?

These are microarchitecture design decisions.

How Much Work in One Clock Cycle?

Consider an instruction such as

```
LDR R1, R2, #5
```

It requires several operations:

Fetch → **Decode** → **Address** → **Memory** → **Write**

Should all these operations happen in

One Clock Cycle

or

Multiple Clock Cycles ?

This is a microarchitecture design choice.

Single-Cycle vs. Multi-Cycle Microarchitecture

Single-Cycle

One instruction completes
in **one clock cycle**

Fetch → Decode → Execute

Longer clock cycle

Multi-Cycle

One instruction completes
in **multiple clock cycles**

Fetch → Decode → ...

Can use a shorter clock period

Same ISA, different microarchitectures

Single-Cycle Execution

In a single-cycle design,

One instruction completes in one clock cycle

For example, LDR requires:

PC → **Memory** → **Registers** → **ALU** → **Memory** → **Register**

All required operations must finish before the next clock edge.

$$T_{\text{clock}} \geq T_{\text{longest instruction path}}$$

Problem with Single-Cycle Design

Different instructions require different amounts of work.

ADD

Registers $\rightarrow$ ALU $\rightarrow$ Register

LDR

Registers $\rightarrow$ ALU $\rightarrow$ Memory $\rightarrow$ Register

But every instruction uses the **same clock period**.

T_{clock} = time required by the slowest instruction

Short instructions must wait for the long clock cycle.

Multi-Cycle Execution

Instead of completing the entire instruction in one cycle,

Divide instruction execution into multiple cycles

For LDR:

Cycle 1: Fetch



Cycle 2: Decode



Cycle 3: Compute Address



Cycle 4: Read Memory

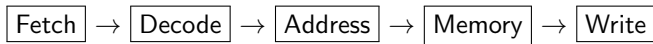


Cycle 5: Write Register

One instruction $\Rightarrow$ Multiple clock cycles

Multi-Cycle Control Requires an FSM

Different operations are performed in different clock cycles.



The processor must remember:

Which step of the instruction is currently being executed?

Each step → FSM State

FSM State → Control Signals

Microarchitecture Is a Design Choice

The **ISA specifies what** the processor must do.

The **microarchitecture decides how** to do it.

Single-Cycle

- One instruction / cycle
- Long clock period
- Simpler sequencing

Multi-Cycle

- Multiple cycles / instruction
- Potentially shorter clock period
- Hardware can be reused

Same ISA can have different microarchitectures
--

From FSM to a Programmable Computer

Hardwired FSM + Datapath → controls a fixed behavior



Datapath + Control FSM → executes operations



Instructions stored in memory → programmable behavior



Processor

ISA = What the machine provides

Microarchitecture = How the machine implements it